



Intro

Smoothstep is one of the most frequently used functions during procedural texturing and modeling, but shows up in many other areas of Computer Graphics as well, as a way to interpolate data with local support (I don't consider functions like `tanh()` or other sigmoids as Smoothsteps if they extend to infinity). Most shading languages provide a standard implementation of smoothstep that is based on the cubic $x^2(3-2x)$ which is quick to evaluate, but other alternatives exist that I sometimes find myself using in some situations.

Some of those alternatives are more convenient, depending on the context. For example the "Quartic Polynomial" below only uses even powers of x , which is convenient when x is a distance for example and x^2 can indeed be computed with a dot product without square roots. Other variants like the "Quintic Polynomial" and "Rational" below have higher continuity than the default smoothstep so they are great for surfaces from which we need to stitch them together (say in a noise function). Others like the "Piecewise Polynomial" one allow controlling the sharpness of the step and others like the "Cubic Polynomial" allow for easy inversion. Worth noting that quartic polynomial version is not symmetric and that the quintic polynomial doesn't have a closed form inverse. Also, I'm not providing integrals in this article, but I have another article [about the integral of the standard smoothstep](#).

In this article I collected a few smoothstep alternatives that I've derived myself or found around. I am presenting the code for each in an unoptimized form so that the formula is clearer so you can more easily do math with it. Usually you can reuse terms and chope a few cycles, but that's easy enough and not worth obfuscating the code.

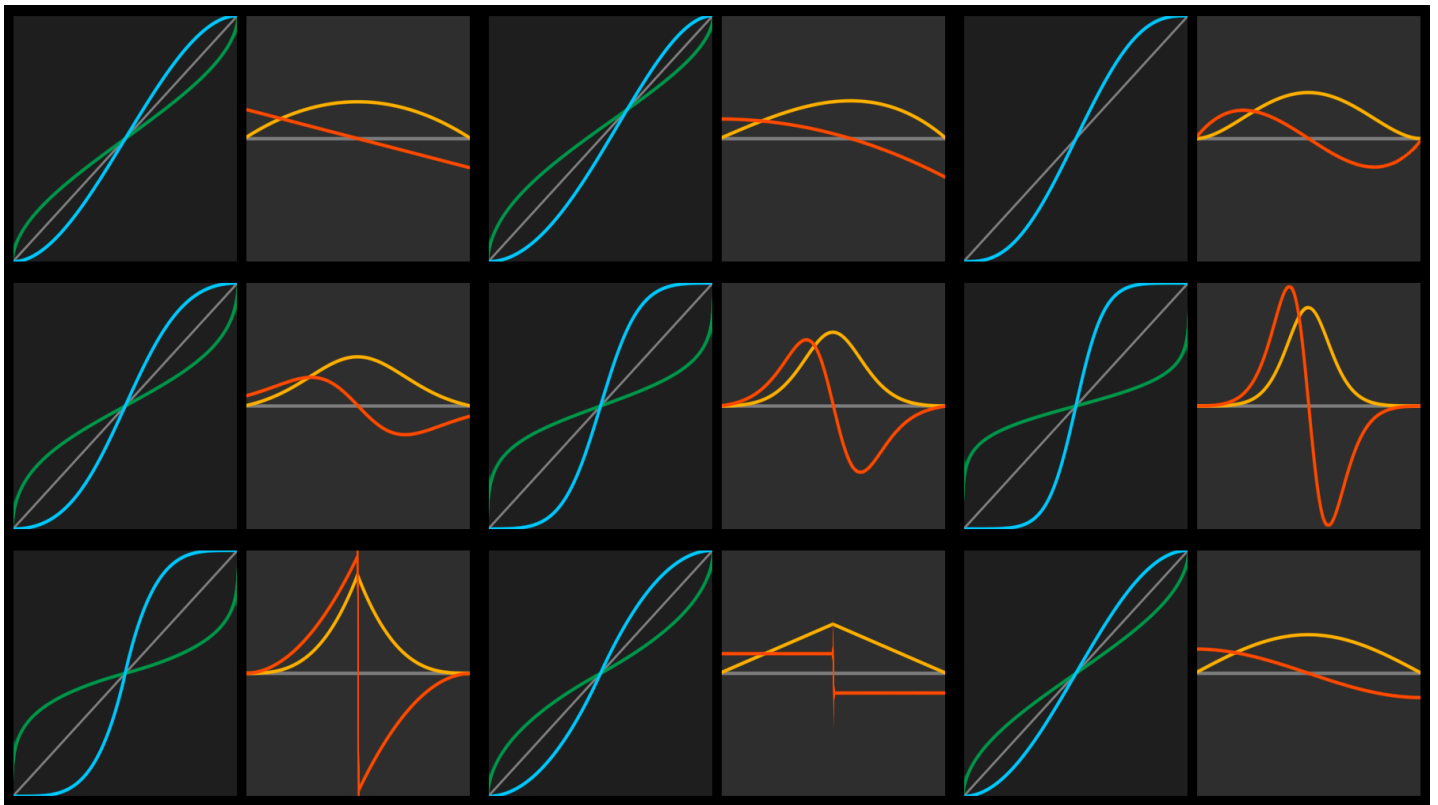
All the smoothsteps below work in the 0 to 1 domain only. For the general form of `smoothstep(a,b,x)` in the interval $[a, b]$, you can simply remap x to $(x-a)/(b-a)$ and clamp to the 0 to 1 range before calling the canonical smoothstep of your choice.

You can see all these smoothsteps, their inverses and derivatives together with the code that implements them in this realtime shader: <https://www.shadertoy.com/view/st2BRd>

Name	Continuity	Code	Inverse
Cubic Polynomial	C1	<pre>float smoothstep(float x) { return x*x*(3.0-2.0*x); }</pre>	<pre>float inv_smoothstep(float x) { return 0.5-sin(asin(1.0-2.0*x)/3.0); }</pre>
Quartic Polynomial	C1	<pre>float smoothstep(float x) { return x*x*(2.0-x*x); }</pre>	<pre>float inv_smoothstep(float x) { return sqrt(1.0-sqrt(1.0-x)); }</pre>
Quintic Polynomial	C2	<pre>float smoothstep(float x) { return x*x*x*(x*(x*6.0-15.0)+10.0); }</pre>	None

Quadratic Rational	C1	<pre>float smoothstep(float x) { return x*x/(2.0*x*x-2.0*x+1.0); }</pre>	<pre>float inv_smoothstep(float x) { return (x-sqrt(x*(1.0-x)))/(2.0*x-1.0); }</pre>
Cubic Rational	C2	<pre>float smoothstep(float x) { return x*x*x/(3.0*x*x-3.0*x+1.0); }</pre>	<pre>float inv_smoothstep(float x) { float a = pow(x,1.0/3.0); float b = pow(1.0-x,1.0/3.0); return a/(a+b); }</pre>
Rational	C(n-1)	<pre>float smoothstep(float x, float n) { return pow(x,n)/(pow(x,n)+pow(1.0-x,n)); }</pre>	<pre>float inv_smoothstep(float x, float n) { return smoothstep(x, 1.0/n); }</pre>
Piecewise Quadratic	C1	<pre>float smoothstep(float x) { return (x<0.5) ? 2.0*x*x: 2.0*x*(2.0-x)-1.0; }</pre>	<pre>float inv_smoothstep(float x) { return (x<0.5) ? sqrt(0.5*x): 1.0-sqrt(0.5-0.5*x); }</pre>
Piecewise Polynomial	C(n-1)	<pre>float smoothstep(float x, float n) { return (x<0.5) ? 0.5*pow(2.0* x, n): 1.0-0.5*pow(2.0*(1.0-x), n); }</pre>	<pre>float inv_smoothstep(float x, float n) { return (x<0.5) ? 0.5*pow(2.0* x, 1.0/P): 1.0-0.5*pow(2.0*(1.0-x),1.0/P); }</pre>
Trigonometric	C1	<pre>float smoothstep(float x) { return 0.5-0.5*cos(PI*x); }</pre>	<pre>float inv_smoothstep(float x) { return acos(1.0-2.0*x)/PI; }</pre>

Below goes a visual comparison of the different smoothsteps above in blue color and their inverse in green. On the right side you can see the first and second derivatives of each smoothstep in yellow and red respectively. As you can see, those I marked as C2 in the table above have second derivative that evaluates to zero on the edges of the 0 to 1 interval, since that ensures that adjacent smoothsteps will connects smoothly to each other. Top row is polynomial smoothsteps, middle row is rationals, bottom row is piecewise polynomials and trigonometric at the bottom right.



As a side note, if you ever need the inverse of the cubic rational smoothstep, instead of solving through the general inverse you can try solving the cubic equation directly as shown in [this article](#), which gives an alternative formulation for it:

```
float inv_smoothstep( float x )
{
    float w=2.0*sqrt(x*(1.0-x));
    float t=(x*(3.0-2.0*x)-1.0)/(w*(1.0-x));
    return x-w*sinh(asinh(t)/3.0);
}
```

inigo quilez - learning computer graphics since 1994